

Progress Report: An Agent-Routed, Memory-Augmented Annotation Workflow for Biomedical Segmentation

Shuheng (Andy) Cao

s8cao@ucsd.edu

1 Problem and Goal

Biomedical segmentation tools fall into two camps that rarely meet in the same workflow: *interactive editors* (3D Slicer, ITK-SNAP, QuPath) that give clinicians fine manual control but no model-assisted priors, and *model-centric pipelines* (MedSAM, MedSAM2, LangSAM, Cellpose) that produce strong masks but offer little transparency, auditability, or reuse of prior prompts. The result is annotation sessions in which the user cannot see *why* a particular backend was chosen, cannot reuse a useful bbox or text prompt across similar slices, and cannot reproduce a session from its exported masks.

This project builds a lightweight desktop and headless workflow that combines manual contouring, classical baselines, and optional foundation-model backends behind a single API, with three properties that distinguish it from existing tools: (1) a transparent *agentic router* that selects a backend from prompt and image cues and logs its reasoning; (2) a *geometry memory* that suggests prior prompts/masks without overwriting user edits; and (3) per-session *provenance sidecars* so every exported mask carries its route, runtime, and prompt context.

2 Progress Since Proposal

2.1 System architecture is implemented end-to-end

The full software path runs today in lightweight mode (no heavy checkpoints required). Figure 1 shows the realised architecture; every block in the diagram corresponds to a working Python module in the repository (66 source files, 17 test modules, `pytest -q` passing on Windows 11/Python 3.11). Concretely, the following pieces are working and exercised by tests:

- **Unified backend API** (`segmentation/backends/`) with mock, classical (OpenCV watershed/levelset), Cellpose, and external-bridge wrappers for LangSAM, MedSAM, MedSAM2.
- **Deterministic agentic router** (`agent/router.py`, `agent/workflow.py`) that ranks candidate backends from prompt tokens, image dimensionality, and bbox/seed presence, and emits a structured `RoutingDecision` record (selected backend, ranked list, fallback history, prompt interpretation).

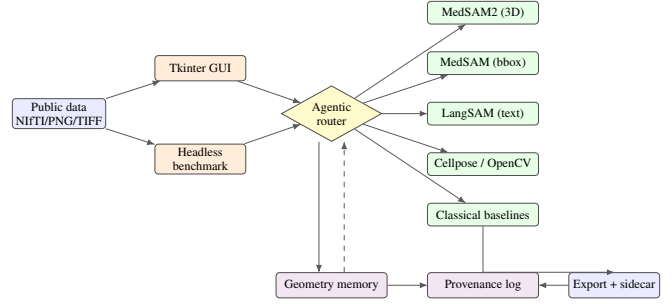


Figure 1: Implemented architecture. Solid arrows are runtime calls; the dashed arrow is the memory→router suggestion path. Heavy backends (MedSAM/MedSAM2/LangSAM) are accessed via external command bridges so the lightweight install runs without a GPU.

- **Memory store** (`memory/`) that records prompt geometry and result summaries without raw pixels by default, so sessions can be reused across similar slices.
- **Provenance sidecars** (`provenance/`) and a python `-m app.provenance.inspect` CLI for review.
- **Headless benchmark runner** (`benchmarks/run_benchmark.py`) that emits `metrics.csv`, `routing-decisions.jsonl`, per-case provenance, and `report.md`.
- **Tkinter GUI** (`python_app/main.py`) that routes through the same workflow and was launched and verified on the development machine.

2.2 Resources secured

LaTeX/MiKTeX, Python 3.11, OpenCV, NumPy/SciPy, and Cellpose are installed locally; per-backend isolated environments live under `.modelenvs/cellpose/` and `.modelenvs/langsam/` to keep heavy dependencies off the main interpreter. Public dataset targets are scoped (MSD subset for 3D medical, Cellpose/BBBC for microscopy) with manifest schemas checked by `app.configs.validate` and a preflight gate (`app.experiments.preflight`) that refuses to launch a real benchmark unless paths, masks, and licenses are present.

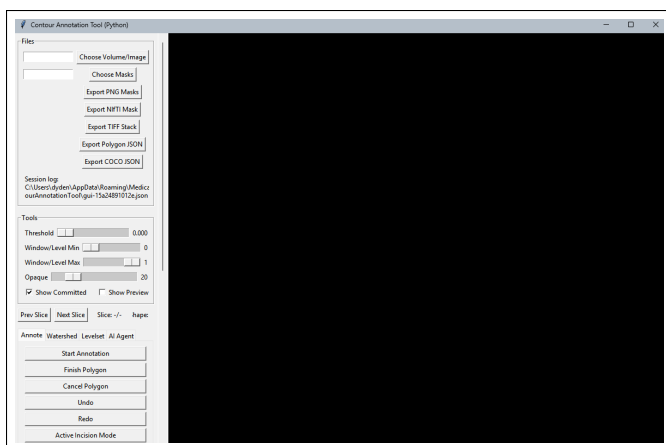


Figure 2: Live Tkinter annotator launched on May 12, 2026. Left column: file I/O, multi-format mask export (PNG / NIfTI / TIFF / polygon JSON / COCO JSON), the auto-generated session-log path, intensity / window-level / slice tools, and tool tabs (Annotate / Watershed / Levelset / AI Agent). The dark region on the right is the image canvas before a volume is loaded.

2.3 Working GUI prototype

Figure 2 is a screenshot of the running Tkinter annotator (python python_app/main.py) on the development machine. The left panel exposes file I/O, the five export formats wired up so far (PNG, NIfTI, TIFF, polygon JSON, COCO JSON), intensity windowing / slice navigation, and four tool tabs — manual contouring, classical watershed, level-set, and the *AI Agent* tab that calls into the router shown in Fig. 1. The session-log line under “Files” is the per-session provenance JSONL emitted automatically.

2.4 Initial quantitative results (synthetic smoke only)

Figure 3 reports Dice scores from the synthetic smoke benchmark (the only quantitative results currently authorised by the project’s claim policy — real public-dataset numbers remain NOT.RUN). The router correctly sends the 2D synthetic microscopy case to the Cellpose backend and the 3D synthetic organ case to the classical fallback, demonstrating that prompt/image cues flow through the router into backend selection as designed. The mock-only ablation isolates router overhead and confirms the routing layer adds < 1 ms per case.

3 What Remains

The engineering scaffold is in place; the remaining work is *evidence*, not infrastructure. Concrete next steps, in order: (1) download the MSD Spleen subset and one Cellpose split, fill `configs/datasets.local.yaml`, and run the real-data benchmark templates already wired up;

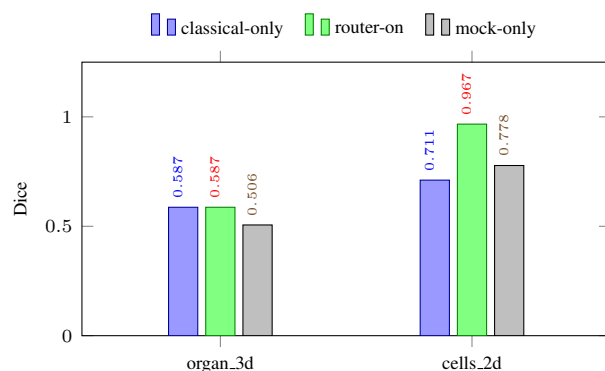


Figure 3: Synthetic smoke Dice from `results/synthetic_smoke/metrics.csv` and `results/mock_ablation/`. Router-on selects CELLPOSE for the 2D microscopy case (Dice 0.967) and CLASSICAL for the 3D organ case (Dice 0.587), matching the routing rules in `agent/router.py`. Numbers are software-path validation, not biomedical accuracy claims.

(2) run baseline exports from 3D Slicer and Cellpose using the protocols in `baseline_protocols/` for a head-to-head comparison; (3) execute the brief usability/time pilot (`docs/usability_study_protocol.md`) on 3–5 users; and (4) produce the camera-ready architecture and results figures from the same scaffolds used here.

Risks. The largest remaining risk is heavy-backend setup on a machine with limited GPU memory; the bridge protocol (`.npz` in / `.npz` out, JSON request file) was chosen specifically so the GUI keeps working when a checkpoint is missing, and the synthetic smoke path validates the router/memory/provenance layers independently of any model weights.